Тестовое задание для C++ разработчика [Middle]

Высокопроизводительный канал сжатых данных

1. Цель

Целью данного задания является разработка и реализация приложения на C++ для симуляции передачи данных между двумя промышленными устройствами по узкому каналу связи. Вы должны реализовать систему, которая считывает данные из бинарного файла, сжимает их, передает через канал связи с ограниченным размером и восстанавливает их на принимающей стороне. При передаче допускается разность между исходными и восстановленными значениями, т.е. возможна потеря данных.

Это задание предназначено для оценки ваших навыков в многопоточности и межпроцессном взаимодействии (IPC), сжатии данных, оптимизации и проектировании систем на C++.

2. Сценарий

Представьте два отдельных **устройства**: **Производитель** и **Потребитель**.

- **Производитель** должен читать исходный файл, сжимать его содержимое и паралельно отправлять сжатые данные потребителю.
- **Потребитель** должен параллельно получать сжатые данные и декомпрессировать их для восстановления исходных данных. Связь между этими двумя процессами ограничена **каналом связи**, реализованным через общую разделяемую память (shared memory).

3. Технические ограничения и условия

- **Язык:** C++ (14-20 стандарт)
- **Платформа:** Решение будет протестировано на Linux (Debian-based OS)
- **Требования к сборке:** CMake; Clang; LLD

- **Зависимости:** Вы можете использовать известные вам библиотеки с открытым исходным кодом. Не рассматриваются решения с использованием фреймворка Qt.
- **Формат файла:** Файл представляет собой бинарное представление чисел с плавающей точкой (32-бит) с разделителем `\n` размером не более 50Mb.
- **Ширина канала связи:** Канал связи являющийся сегментом общей разделяемой памяти строго ограничен **256 байтами**.
- **Процессы:** Вы должны реализовать **Производителя** и **Потребителя** как отдельные процессы (два исполнительных файла).

4. Критерии приемки

Основными критериями успеха являются **работоспособность** и **соответствиие техническому заданию**. Будет плюсом если вы сможете подобрать решение, в котором общее время, затраченное на весь конвейер (сжатие + передача + декомпрессия), будет **меньше**, чем время, которое потребовалось бы для передачи несжатого исходного файла через тот же 256-байтный канал общей памяти, с минимальными допущеными потерями данных. Дополнительным Вы можете реализовать графический интерфейс (например ImGui или SDL).

Вы должны включить простой способ измерения и отображения:

1. Время сжатия и передачи данных от Производителя к Потребителю.
2. Время для передачи данных без сжатия.
3. Процент сжатия данных.
4. Процент потери данных.

5. Ожидаемые результаты

1. **Исходный код:** Ссылка на опубликованный проект (GitHub, GitLab, пр.) или tar.gz архив
2. **Инструкции:** Краткий файл README.md.
3. **Документация:** Короткий раздел в вашем README.md, объясняющий:
   - Выбор алгоритма сжатия; ход мыслей в процессе разработки.
   - Метод синхронизации общей памяти.
   - Компромиссы, на которые вы пошли для оптимизации.

Версия #16
Admin создал 22 сентября 2025 14:18:36
Admin обновил 3 апреля 2026 12:48:02